

Fájlkezelés C++-ban: olvasás és írás

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

A jegyzet két részből áll. Az 1-4. fejezetben megtanulod, hogyan olvas a programod adatokat fájlból, és hogyan írsz eredményt fájlba. Az 5-8. fejezet összetett feladatokat tartalmaz, amelyekben az adatok már fájlból érkeznek.

- **Ellenőrzés.** Minden feladatnál ott van, mit kell kapnod. Csak akkor lépj tovább, ha egyezik.
- **Hibakereső.** Ha valami nem működik, a jegyzet végén lévő táblázat segít.
- **Segítség.** Az 5-8. fejezet feladataihoz a jegyzet legvégén találsz útbaigazítást. Csak akkor lapozz oda, ha legalább öt percig próbálkoztál.

A feladatokhoz kilenc adatfájlt kaptál:

Fájl	Mi van benne?	Fejezet
szamok.txt	néhány egész szám	1. és 4.
nevsor.txt	nevek, soronként egy	2.
diakok.txt	diákok adatai: <code>nev;kor;atlag</code>	2. és 4.
termek.txt	termékek, az első sorban a darabszám	3.
meresek.txt	mérések, az első sorban fejléc	3.
lepesek.txt	egy hónap napi lépésszámai	5.
hivasok.txt	egy ügyfélszolgálat hívásai	6.
verseny.txt	egy futóverseny eredményei	7.
vasarlasok.txt	egy büfé heti forgalma	8.

1. Olvasás fájlból

Fájlból ugyanúgy olvasol, mint a billentyűzetről. Csak előbb meg kell nyitnod a fájlt, és a `std::cin` helyett a megnyitott fájl nevét írod.

Kísérlet - hol fut a programom?

A program csak akkor találja meg az adatfájlt, ha az a megfelelő mappában van. Hogy melyik az, azt ezzel a programmal derítheted ki:

```
#include <fstream>

int main() {
    std::ofstream ki("itt_vagyok.txt");
    ki << "Ebben a mappában fut a programod." << std::endl;
    return 0;
}
```

Futtasd, aztán keresd meg a gépeden az `itt_vagyok.txt` fájlt (a projekted mappájában vagy annak valamelyik almappájában lesz). Ahol megtaláltad, **oda másold be mind a kilenc adatfájlt.**

Próbáld ki - az első beolvasás

A `szamok.txt` tartalma (nyisd meg, és nézd meg):

```
12 7 25 4
18 31
9 16 22 3 40 5
```

```
#include <fstream>
#include <iostream>

int main() {
    std::ifstream be("szamok.txt");

    int x;
    be >> x;
    std::cout << "Az első szám: " << x << std::endl;
    return 0;
}
```

Nézzük meg az új részeket:

- `#include <fstream>` - ez kell a fájlkezeléshez.
- `std::ifstream be("szamok.txt");` - megnyitja a fájlt **olvasásra**. A `be` egy változó neve, te választod.
- `be >> x;` - ugyanaz, mint a `std::cin >> x;`, csak a szám a fájlból érkezik.

Kísérlet - ha a program nem találja a fájlt

Írd el a fájl nevét a programban: `"szamok.txt"`. Futtasd. Kaptál hibaüzenetet? Mit írt ki a program?

A C++ nem szól, ha a fájlt nem sikerült megnyitni: a beolvasás egyszerűen nem működik, a változóban pedig értelmetlen érték marad. Ezért a megnyitás után mindig ellenőrizd:

```
std::ifstream be("szamok.txt");
if (!be.is_open()) {
    std::cout << "Nem sikerült megnyitni a fájlt!" << std::endl;
    return 1;
}
```

Itt a `return 1;` azonnal befejezi a programot. Próbáld ki az elírt fájlnévvel, aztán javítsd vissza.

Figyelem! A megnyitás után mindig ellenőrizd az `is_open()` függvénnyel, hogy sikerült-e. Ha nem, két dolgot nézz meg: jól írtad-e a fájl nevét, és jó mappában van-e a fájl.

Tippelj - hol tart az olvasás?

```
int a, b, c;
be >> a >> b >> c;
std::cout << a + b + c << std::endl;

int d;
be >> d;
std::cout << d << std::endl;
```

A fájlban az olvasás mindig ott folytatódik, ahol az előző abbamaradt.

Próbáld ki - az összes szám beolvasása

Általában nem tudod előre, hány adat van a fájlban. Ilyenkor addig olvasol, amíg sikerül:

```
#include <fstream>
#include <iostream>
#include <vector>

int main() {
    std::ifstream be("szamok.txt");
    if (!be.is_open()) {
        std::cout << "Nem sikerult megnyitni a fajlt!" << std::endl;
        return 1;
    }

    std::vector<int> szamok;
    int x;
    while (be >> x) {
        szamok.push_back(x);
    }
    be.close();

    std::cout << szamok.size() << " szamot olvastam be." << std::endl;
    return 0;
}
```

- A `while (be >> x)` addig ismételi, amíg sikerül egy újabb számot beolvasni. A fájl végén a beolvasás nem sikerül, és a ciklus leáll. Ugyanezt láttad a `while (ss >> szo)` ciklusnál.
- A `be.close();` bezárja a fájlt, ha már nincs rá szükség.

Figyeld meg a fájl tartalmát: a számok három sorban, szabálytalanul állnak. Zavarta ez a `>>` műveletet?

Mostantól az adatok a `szamok` vektorban vannak, és mindent megtehetsz velük, amit a vektoros jegyzetben tanultál.

Feladatok

1.1. feladat ★☆☆ - Az első három

Írd ki a `szamok.txt` első három számát egy sorba.

Ellenőrzés: 12 7 25

1.2. feladat ★☆☆ - Darab és összeg

Olvasd be az összes számot egy vektorba. Írd ki, hány szám van, és mennyi az összegük.

Ellenőrzés: 12 szám, az összegük 192.

1.3. feladat ★☆☆ - Legnagyobb

Írd ki a fájl legnagyobb számát.

Ellenőrzés: 40.

1.4. feladat ★☆☆ - Párosak

Hány páros szám van a fájlban?

Ellenőrzés: 6.

2. Sorok olvasása

Ha egy sor összetartozó adatokat tartalmaz, soronként kell olvasnod. Erre a `std::getline` való, ugyanúgy, mint a billentyűzetnél.

Tippelj - szavak vagy sorok?

A `nevsor.txt` tartalma:

```
Kiss Anna  
Nagy Bence  
Toth Csilla Reka  
Szabo Dani  
Varga Emma  
Kovacs Ferenc Pal
```

Hányat számol ez a programrészlet? (A fájl megnyitását és ellenőrzését az előző fejezet szerint írd elé.)

```
std::string szo;  
int db = 0;  
while (be >> szo) {  
    db++;  
}  
std::cout << db << std::endl;
```

Próbáld ki - soronként

```
std::string sor;  
int db = 0;  
while (std::getline(be, sor)) {  
    std::cout << db + 1 << ". sor: " << sor << std::endl;  
    db++;  
}  
std::cout << db << " sor volt a fajlban." << std::endl;
```

A `std::getline(be, sor)` egy egész sort olvas be a fájlból. A `while` addig ismételi, amíg van még sor.

Próbáld ki - adatsorokból structok

A `diakok.txt` minden sora egy diák adatait tartalmazza `nev;kor;atlag` formában. A sorból structot készítő `feldolgoz` függvényt a struct jegyzetből ismered:

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

struct Diak {
    std::string nev;
    int kor;
    double atlag;
};

Diak feldolgoz(std::string sor) {
    std::stringstream ss(sor);
    std::string korSzoveg, atlagSzoveg;

    Diak d;
    std::getline(ss, d.nev, ';');
    std::getline(ss, korSzoveg, ';');
    std::getline(ss, atlagSzoveg, ';');
    d.kor = std::stoi(korSzoveg);
    d.atlag = std::stod(atlagSzoveg);
    return d;
}

int main() {
    std::ifstream be("diakok.txt");
    if (!be.is_open()) {
        std::cout << "Nem sikerült megnyitni a fajlt!" << std::endl;
        return 1;
    }

    std::vector<Diak> diakok;
    std::string sor;
    while (std::getline(be, sor)) {
        diakok.push_back(feldolgoz(sor));
    }
    be.close();

    std::cout << diakok.size() << " diák adatait olvastam be." << std::endl;
    std::cout << "Az első: " << diakok[0].nev << std::endl;
    return 0;
}
```

Az előző feladatlapon a sorokat beírtuk a programba. Most ugyanilyen sorok fájlból érkeznek – a `feldolgoz` függvényen semmit nem kellett változtatni.

Tipp: Beolvasás után mindig írd ki, hány elemet olvastál be, és mi az első elem. Ha ez a kettő stimmel, a beolvasásod nagy valószínűséggel jó.

Feladatok

2.1. feladat ★☆☆ - Névsor

Olvasd be a `nevsor.txt` sorait egy vektorba. Írd ki, hány név van benne, és melyik a leghosszabb név.

Ellenőrzés: 6 név; a leghosszabb `Kovacs Ferenc Pal` (17 karakter).

2.2. feladat ★☆☆ - Diákok

A fenti program segítségével számold ki a `diakok.txt` diákjainak átlagos tanulmányi átlagát.

Ellenőrzés: 10 diák, az átlaguk 3.92.

2.3. feladat ★☆☆ - A legjobb

Kinek a legjobb az átlaga?

Ellenőrzés: Toth Csilla (4.9).

2.4. feladat ★☆☆ - Nagykorúak

Írd ki a nagykorú (legalább 18 éves) diákok nevét.

Ellenőrzés: Toth Csilla, Kovacs Feri, Balogh Iren.

3. Két gyakori fájlforma

Az adatfájlok első sora gyakran nem adat. Vagy azt mondja meg, hány adatsor következik, vagy fejléc: az oszlopok neve.

Próbáld ki - darabszám az első sorban

A `termek.txt` tartalma:

```
6
fuzet;350;12
toll;180;30
radir;120;8
vonalzo;420;5
ceruza;90;40
korzo;990;3
```

```

#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

struct Termek {
    std::string nev;
    int ar;
    int darab;
};

Termek feldolgoz(std::string sor) {
    std::stringstream ss(sor);
    std::string arSzoveg, darabSzoveg;

    Termek t;
    std::getline(ss, t.nev, ';');
    std::getline(ss, arSzoveg, ';');
    std::getline(ss, darabSzoveg, ';');
    t.ar = std::stoi(arSzoveg);
    t.darab = std::stoi(darabSzoveg);
    return t;
}

int main() {
    std::ifstream be("termekek.txt");
    if (!be.is_open()) {
        std::cout << "Nem sikerult megnyitni a fajlt!" << std::endl;
        return 1;
    }

    int db;
    be >> db;
    be.ignore();

    std::vector<Termek> termekek;
    for (int i = 0; i < db; i++) {
        std::string sor;
        std::getline(be, sor);
        termekek.push_back(feldolgoz(sor));
    }
    be.close();

    std::cout << termekek.size() << " termek adatait olvastam be." << std::endl;
    return 0;
}

```

Az első sort a `be >> db;` olvassa be, utána `for` ciklus olvas pontosan ennyi sort. A `be.ignore();` ismerős a sztringes jegyzetből: a szám utáni sorvéget ugorja át.

Kísérlet - ha kimarad az ignore

Töröld ki a `be.ignore();` sort, és futtasd. Mi történik? Keresd meg a hibaüzenetet a Hibakeresőben, aztán írd vissza a sort.

Próbáld ki - fejléc az első sorban

A `meresek.txt` első néhány sora:

```
nap;homerseklet;csapadek
1;12.5;0
2;14.1;0
3;9.8;4.2
```

Az első sor nem adat, ezért beolvassuk, de nem dolgozzuk fel:

```
std::string fejléc;
std::getline(be, fejléc);

std::string sor;
while (std::getline(be, sor)) {
    // itt dolgozd fel a sort
}
```

Feladatok

3.1. feladat ★☆☆ - Készlet

Mennyit ér a `termekek.txt` teljes készlete (ár szorozva darabszámmal, összesen)?

Ellenőrzés: 6 termék, a készlet értéke 19230.

3.2. feladat ★☆☆ - Legdrágább

Melyik a legdrágább termék?

Ellenőrzés: `korzo` (990).

3.3. feladat ★☆☆ - Mérések

Írj `Meres` structot (`nap`, `homerseklet`, `csapadek`) és hozzá `feldolgoz` függvényt. Olvasd be a `meresek.txt` adatait a fejléc kihagyásával, és írd ki az átlaghőmérsékletet.

Ellenőrzés: 14 mérés, az átlaghőmérséklet 11.5643.

3.4. feladat ★☆☆ - Csapadék

Hány napon esett csapadék, és mennyi esett összesen? Mi történik, ha a fejléct elfelejtet kihagyni?

Ellenőrzés: 6 napon, összesen 26.

4. Írás fájlba

Fájlba ugyanúgy írsz, mint a képernyőre. Csak előbb meg kell nyitnod a fájlt írásra, és a `std::cout` helyett a megnyitott fájl nevét írod.

Próbáld ki

```
#include <fstream>
#include <iostream>

int main() {
    std::ofstream ki("eredmeny.txt");

    ki << "Ez az elso sor." << std::endl;
    ki << "Harom meg negy az " << 3 + 4 << std::endl;
    ki.close();

    std::cout << "Kesz, nezd meg a fajlt!" << std::endl;
    return 0;
}
```

- `std::ofstream ki("eredmeny.txt");` – megnyitja a fájlt **írásra**. Ha a fájl még nem létezik, létrehozza.
- `ki << ...` – ugyanaz, mint a `std::cout << ...`, csak a szöveg a fájlba kerül.
- `ki.close();` – bezárja a fájlt. Ezt ne hagyd el: a bezárás biztosítja, hogy minden a fájlba kerüljön.

Keresd meg az `eredmeny.txt` fájlt az adatfájlok mellett, és nyisd meg.

Kísérlet - mi lesz a régi tartalommal?

Írd át a programban az első sort másra, és futtasd újra. Mi van most a fájlban: a régi és az új szöveg is, vagy csak az új?

Figyelem! Az `std::ofstream` megnyitáskor törli a fájl korábbi tartalmát. Soha ne nyisd meg írásra azt a fájlt, amelyből az adataidat olvasod!

Próbáld ki - olvasás és írás egy programban

Ez a programrészlet beolvassa a `szamok.txt` számait, és mindegyik kétszeresét kiírja a `duplak.txt` fájlba:

```
std::ifstream be("szamok.txt");
std::ofstream ki("duplak.txt");

int x;
while (be >> x) {
    ki << x * 2 << std::endl;
}

be.close();
ki.close();
```

Egy programban több fájl is nyitva lehet egyszerre; mindegyiknek külön neve van. Ugyanazt az eredményt kiírhatod a képernyőre és fájlba is.

Feladatok

4.1. feladat ★☆☆ - Névjegy

Írj programot, amely a `nevjegy.txt` fájlba kiírja a nevedet és az osztályodat, két külön sorba.

Ellenőrzés: nyisd meg a fájlt; két sor legyen benne.

4.2. feladat ★☆☆ - Négyzetszámok

Írd ki a `negyzetek.txt` fájlba a számokat 1-től 20-ig, mindegyik mellé a négyzetét, szóközzel elválasztva.

Ellenőrzés: 20 sor; az első `1 1`, az utolsó `20 400`.

4.3. feladat ★☆☆ - Csak a párosak

Olvasd be a `szamok.txt` számait, és a párosakat írd ki a `paros.txt` fájlba, soronként egyet.

Ellenőrzés: a fájl hat sora: 12, 4, 18, 16, 22, 40.

4.4. feladat ★☆☆ - Kitűnők

Olvasd be a `diakok.txt` adatait, és írd ki a `kitunok.txt` fájlba azoknak a nevét és átlagát, akiknek az átlaga legalább 4.5.

Ellenőrzés: három sor: `Kiss Anna 4.5`, `Toth Csilla 4.9`, `Horvath Greta 4.7`.

4.5. feladat ★☆☆ - Oda-vissza

Olvasd vissza a 4.2. feladatban készített `negyzetek.txt` fájlt egy másik programmal, és add össze a négyzetszámokat (a sorok második számát).

Ellenőrzés: 2870.

5. Lépésszámláló

Amit gyakorolsz: számok beolvasása vektorba, alapminták, szomszédos elemek, kiírás fájlba.

A `lepesek.txt` egy okosóra októberi adatait tartalmazza: minden sorban egy nap lépésszáma áll, a hónap első napjától az utolsóig.

5.1. feladat ★☆☆ - Beolvasás

Olvasd be az adatokat egy vektorba. Írd ki, hány nap adata van meg, és mennyi az első és az utolsó napi érték.

Ellenőrzés: 31 nap; az első érték 5635, az utolsó 13944.

5.2. feladat ★☆☆ - Összesen és átlagosan

Hány lépést tett meg a hónapban összesen, és mennyi a napi átlag?

Ellenőrzés: összesen 337918 lépés, az átlag 10900.6.

5.3. feladat ★☆☆ - A legaktívabb nap

Hányadik napon lépett a legtöbbet, és mennyit?

Ellenőrzés: a 4. napon, 15509 lépést.

5.4. feladat ★☆☆ - Tízezres napok

Hány napon érte el a 10 000 lépést?

Ellenőrzés: 21 napon.

5.5. feladat ★☆☆ - Jó napok fájlba

Írd ki a `jo_napok.txt` fájlba azokat a napokat, amikor az átlagnál többet lépett: soronként a nap sorszáma és a lépésszám, szóközzel elválasztva.

Ellenőrzés: a fájlban 18 sor van; az első `2 13498`, az utolsó `31 13944`.

5.6. feladat ★☆☆ - Hétvége

Október 1-je csütörtökre esett. Mennyi a hétvégi napok (szombat és vasárnap) átlaga, és mennyi a hétköznapiaké?

Ellenőrzés: 9 hétvégi nap, átlaguk 10562; 22 hétköznapi, átlaguk 11039.1.

5.7. feladat ★★★ - Leghosszabb sorozat

Hány napig tartott a leghosszabb olyan sorozat, amikor minden egymást követő napon megvolt a 10 000 lépés? Melyik napon kezdődött?

Ellenőrzés: 5 napig tartott, és a 4. napon kezdődött.

5.8. feladat ★★★ - A legjobb hét

Melyik hét egymást követő napon lépett összesen a legtöbbet? Add meg a kezdőnap sorszámát és a hét napi összeget.

Ellenőrzés: a 2. napon kezdődő hét nap, összesen 89326 lépés.

6. Ügyfélszolgálat

Amit gyakorolsz: több szám egy sorban, struct, számolás időpontokkal, számláló vektor, szomszédos elemek.

A **hivasok.txt** egy telefonos ügyfélszolgálat egy napi hívásait tartalmazza, időrendben. Egy sor egy hívás: hat szám, szóközzel elválasztva. Az első három a hívás kezdete (óra, perc, másodperc), a második három a vége. Az első három sor:

```
8 1 35 8 13 56
8 13 40 8 21 40
8 21 38 8 22 38
```

6.1. feladat ★☆☆ - Másodpercben

Írj `int mpben(int ora, int perc, int mp)` függvényt, amely megadja, hány másodperc telt el éjfél óta.

Ellenőrzés: `mpben(8, 1, 35) → 28895`, `mpben(0, 0, 59) → 59`.

6.2. feladat ★★★ - Beolvasás

Írj `Hivas` structot két mezővel: `kezdet` és `veg` (mindkettő másodpercben), és egy `int hossz()` tagfüggvénnyel. Olvasd be a hívásokat egy vektorba.

Ellenőrzés: 50 hívás; az első hossza 741 másodperc.

6.3. feladat ★☆☆ - A leghosszabb hívás

Hányadik hívás volt a leghosszabb, és meddig tartott? Írd ki percben és másodpercben is.

Ellenőrzés: a 18. hívás, 759 másodperc (12 perc 39 másodperc).

6.4. feladat ★☆☆ - Hívásidők

Mennyi az átlagos hívásidő? Hány hívás tartott 5 percnél tovább? Mennyi az összes beszélgetési idő óra, perc, másodperc formában?

Ellenőrzés: az átlag 268.7 másodperc; 16 hívás volt 5 percnél hosszabb; az összes idő 3 óra 43 perc 55 másodperc.

6.5. feladat ★☆☆ - Óránkénti forgalom

Melyik órában hány hívás kezdődött? Írd ki az eredményt a `forgalom.txt` fájlba (8 óra: 8 hívás formájú sorokkal), és a képernyőre azt, melyik órában kezdődött a legtöbb hívás.

Ellenőrzés: 8 óra: 8, 9 óra: 6, 10 óra: 7, 11 óra: 7, 12 óra: 6, 13 óra: 4, 14 óra: 5, 15 óra: 7. A legtöbb hívás 8 és 9 óra között kezdődött.

6.6. feladat ★☆☆ - Várakozók

Az ügyfélszolgálaton egyetlen ügyintéző dolgozik. Hány hívó volt kénytelen várni, vagyis hány hívás kezdődött úgy, hogy az előző hívás még nem ért véget?

Ellenőrzés: 14.

6.7. feladat ★★★ - A leghosszabb szünet

Mennyi volt a leghosszabb szünet, vagyis a leghosszabb idő egy hívás vége és a következő hívás kezdete között? Melyik két hívás között volt?

Ellenőrzés: 1267 másodperc (21 perc 7 másodperc), a 36. és a 37. hívás között.

6.8. feladat ★★★ - Hosszú hívások fájlba

Írd ki a `hosszu.txt` fájlba az 5 percnél hosszabb hívásokat: soronként a hívás sorszáma, a kezdetének órája és perce, valamint a hossza másodpercben.

Ellenőrzés: 16 sor; az első `1 8 1 741`, az utolsó `49 15 34 564`.

7. Futóverseny

Amit gyakorolsz: *fejléces fájl, struct, keresés, megszámlálás kategóriánként, rendezés, formázott kiírás fájlba.*

A `verseny.txt` egy futóverseny eredményeit tartalmazza. Az első sor fejléc, utána minden sor egy versenyző: rajtszám, név, kategória (**F** férfi, **N** női, **J** junior), és az idő percben és másodpercben. Az első négy sor:

```
rajtszam;nev;kategoria;perc;masodperc
42;Papp Tamas;F;42;42
20;Meszaros Bence;F;48;3
51;Papp Reka;N;52;28
```

7.1. feladat ★★★ - Beolvasás

Írj `Futo` structot az öt mezővel (a kategória legyen `std::string`) és egy `int ido()` tagfüggvénnyel, amely az időt másodpercben adja vissza. Olvasd be a versenyzőket a fejléc kihagyásával.

Ellenőrzés: 36 versenyző; az első Papp Tamas, az ideje 2562 másodperc.

7.2. feladat ★★★ - Időpont szépen

Írj `std::string idoSzoveg(int mp)` függvényt, amely a másodpercben megadott időt `perc:masodperc` formájú szöveggé alakítja. A másodperc mindig két számjegy legyen.

Ellenőrzés: `idoSzoveg(2412) → 40:12`, `idoSzoveg(3243) → 54:03`.

7.3. feladat ★☆☆ - A győztes

Ki nyerte a versenyt, és mennyi idő alatt?

Ellenőrzés: Kiss Petra, 40:12.

7.4. feladat ★☆☆ - Létszámok

Hány versenyző indult az egyes kategóriákban?

Ellenőrzés: F: 15, N: 14, J: 7.

7.5. feladat ★☆☆ - Kategóriagyőztesek

Ki nyert az egyes kategóriákban? Írj hozzá függvényt, amely megkapja a versenyzők vektorát és egy kategóriát, és visszaadja a kategória legjobbját.

Ellenőrzés: F: Molnar Dani (40:18); N: Kiss Petra (40:12); J: Meszaros Reka (46:38).

7.6. feladat ★☆☆ - Átlagidő

Mennyi az átlagos idő? Hány versenyző futott az átlagnál jobbat?

Ellenőrzés: 3105.36 másodperc, vagyis 51:45; 17 versenyző volt ennél gyorsabb.

7.7. feladat ★☆☆ - Keresés

Olvass be egy rajtszámot a billentyűzetről. Írd ki a versenyző nevét, idejét és azt, hányadik helyen végzett összesítésben. Ha nincs ilyen rajtszám, ezt írd ki: **Nincs ilyen rajtszám.**

Ellenőrzés: 71 → Farkas Oliver, 45:11, 10. hely; 52 → Racz Greta, 54:03, 21. hely; 1 → **Nincs ilyen rajtszám.**

7.8. feladat ★★☆☆ - Eredménylista

Írd ki az **eredmeny.txt** fájlba a verseny végeredményét, a leggyorsabbtól a leglassabbig, ilyen sorokkal:
1. Kiss Petra (N) 40:12

Ellenőrzés: 36 sor. Az első három: 1. Kiss Petra (N) 40:12, 2. Molnar Dani (F) 40:18, 3. Kiss Levente (F) 41:29. Az utolsó: 36. Szabo Flora (N) 64:41.

7.9. feladat ★★☆☆ - Dobogósok

Írd ki a **dobogo.txt** fájlba mindhárom kategória első három helyezettjét.

Ellenőrzés: F: Molnar Dani, Kiss Levente, Papp Tamas. N: Kiss Petra, Balogh Lilla, Juhasz Bori. J: Meszaros Reka, Varga Csilla, Kovacs Bence.

8. Iskolai büfé

Amit gyakorolsz: struct, különböző értékek kigyűjtése, összesítés termékenként, rendezés, fájl írása és visszaolvasása. Ez a jegyzet legnehezebb fejezete.

A **vasarlasok.txt** egy iskolai büfé egy heti eladásait tartalmazza. Egy sor egy eladott tétel: a nap, a termék neve, a darabszám és az egységár. Ugyanaz a termék egy napon többször is szerepelhet. Az első négy sor:

```
hetfo;szendvics;5;650
hetfo;alma;4;150
hetfo;kakaos csiga;2;420
hetfo;pogacs;7;280
```

8.1. feladat ★☆☆ - Beolvasás

Írj **Tetel** structot a négy mezővel és egy **int ertek()** tagfüggvénnyel (darabszám szorozva az egységárral). Olvasd be a tételeket.

Ellenőrzés: 56 tétel; az első tétel értéke 3250.

8.2. feladat ★☆☆ - Heti bevétel

Mennyi volt a büfé heti bevétele?

Ellenőrzés: 96810 Ft.

8.3. feladat ★☆☆ - A legnagyobb tétel

Melyik volt a legnagyobb értékű tétel?

Ellenőrzés: hétfőn 9 szendvics, 5850 Ft.

8.4. feladat ★☆☆ - Napi bevétel

Írj **int napiBevetel(std::vector<Tetel> v, std::string nap)** függvényt. Írd ki mind az öt nap bevételeit, és azt, melyik napon volt a legnagyobb.

Ellenőrzés: hetfo 18370, kedd 23590, szerda 13300, csutortok 21320, pentek 20230. A legjobb nap a kedd.

8.5. feladat ★★★ - Termékek

Hányféle terméket árul a büfé? Gyűjtsd ki a termékek nevét úgy, hogy mindegyik csak egyszer szerepeljen.

Ellenőrzés: 8-féle: szendvics, alma, kakaos csiga, pogacsa, muzli szelet, víz, tea, joghurt.

8.6. feladat ★★★ - Összesítés termékenként

Írj `Osszesites` structot: `termek`, `darab`, `bevetel`. Készíts minden termékhez egy összesítést: hány darabot adtak el belőle a héten, és mennyi bevételt hozott. Melyik termékből fogyott a legtöbb darab? Melyik hozta a legtöbb bevételt?

Ellenőrzés: a legtöbb darab a pogácsából fogyott (63); a legtöbb bevételt a szendvics hozta (24050 Ft).

8.7. feladat ★★★ - Összesítés fájlba, és vissza

Írd ki az összesítést az `osszesites.txt` fájlba, bevétel szerint csökkenő sorrendben, `termek;darab;bevetel` formában. Utána ellenőrizd a saját munkádat: egy másik programmal olvasd vissza a fájlt, és add össze a bevételeket.

Ellenőrzés: az első sor `szendvics;37;24050`, az utolsó `tea;25;7500`. A visszaolvasott bevételek összege 96810, vagyis megegyezik a 8.2. feladat eredményével.

8.8. feladat ★★★ - Mi nem fogyott pénteken?

Melyik termékekből nem adtak el egyet sem pénteken?

Ellenőrzés: alma és tea.

Hibakereső: mi lehet a baj?

A fájlkezelés hibái ritkán járnak hibaüzenettel. Keresd meg a táblázatban azt, amit tapasztalsz.

Amit látsz	Mit jelent?	Mit tegyél?
A program lefut, de 0 elemet olvas be, vagy értelmetlen számot ír ki	Nem sikerült megnyitni a fájlt.	Jól írtad a fájl nevét? Jó mappában van? Ellenőrizd az <code>is_open()</code> függvénnyel.
<code>terminate called after throwing an instance of 'std::invalid_argument'</code>	A <code>std::stoi</code> vagy a <code>std::stod</code> olyan szöveget kapott, amely nem szám.	Kihagytad a fejléct? Megvan a <code>be.ignore();</code> a darabszám után? Van üres sor a fájl végén?
Egyel több vagy egyel kevesebb elem lett a vektorban	Az első sort (darabszám vagy fejléc) rosszul kezelted.	Írd ki az első és az utolsó elemet, és hasonlítsd össze a fájllal.
Csak a név első szava került a mezőbe	Szavanként olvastál sor helyett.	<code>be >> ...</code> helyett <code>std::getline(be, sor)</code>
<code>'ifstream' is not a member of 'std'</code>	Hiányzik a fájlkezelés fejléce.	<code>#include <fstream></code>
A kimeneti fájl nem található	A fájl ott jön létre, ahol a program fut.	Ugyanott keresd, ahol az <code>itt_vagyok.txt</code> volt.
A kimeneti fájl üres vagy hiányos	A fájlba nem a megfelelő helyen írtál, vagy nem zártad be.	A <code>ki << ...</code> sor a cikluson belül van? Megvan a <code>ki.close();</code> ?
A adatfájlod kiürült	Megnyitottad írásra (<code>std::ofstream</code>).	Másold be újra az eredeti fájlt, és javítsd a fájlnevet a programban.
A képernyőn megjelenik az eredmény, a fájlban nem	<code>std::cout</code> -ra írtál a fájl helyett.	<code>std::cout << ...</code> helyett <code>ki << ...</code>

Összefoglaló táblázat (puska)

Minden fájlkezelő programba kell: `#include <fstream>`

Minta	Jelentés
<code>std::ifstream be("adat.txt");</code>	fájl megnyitása olvasásra
<code>if (!be.is_open()) { ... }</code>	sikerült-e megnyitni?
<code>be >> x;</code>	egy szám vagy szó beolvasása
<code>while (be >> x) { ... }</code>	olvasás a fájl végéig, számonként vagy szavanként
<code>std::getline(be, sor);</code>	egy egész sor beolvasása
<code>while (std::getline(be, sor)) { ... }</code>	olvasás a fájl végéig, soronként
<code>be >> db; majd be.ignore();</code>	darabszám az első sorban
<code>std::getline(be, fejléc);</code>	fejléc beolvasása és kihagyása
<code>std::ofstream ki("eredmeny.txt");</code>	fájl megnyitása írásra (a régi tartalom törlődik)
<code>ki << x << std::endl;</code>	írás a fájlba
<code>be.close(); és ki.close();</code>	fájl bezárása

Figyelem! Három dolog, amit soha ne felejts el: megnyitás után ellenőrizd az `is_open()` függvénnyel, hogy sikerült-e; beolvasás után írd ki, hány elemet olvastál be; az adatfájlt soha ne nyisd meg írásra.

Segítség a feladatokhoz

Csak akkor olvasd el egy feladat segítségét, ha már legalább öt percig próbálkoztál vele. A segítség nem megoldás: azt mondja meg, merre indulj.

5. fejezet

- **5.1.** Az 1. fejezet programja: `while (be >> x)` és `push_back`.
- **5.2.** Az összeg legyen `double`, hogy az átlag ne egész osztással készüljön.
- **5.5.** Előbb az átlag kell. Utána nyiss meg egy `std::ofstream` fájlt, és a ciklusban a feltételnek megfelelő napokat írd bele: `ki << i + 1 << " " << lepesek[i] << std::endl;`
- **5.6.** Ha az 1. nap csütörtök, akkor a 3. nap szombat, a 4. vasárnap, és ez hétfőnként ismétlődik. A nap sorszámanak 7-tel vett maradéka tehát szombaton 3, vasárnap 4.
- **5.7.** Vezess egy `hossz` számlálót: ha az aznapi érték legalább 10 000, növeld; ha nem, nullázd. Közben jegyezd meg az eddigi legnagyobb hosszt, és azt, hol kezdődött az a sorozat.
- **5.8.** Minden lehetséges kezdőnapra add össze a hét egymást követő nap értékét (ciklus a ciklusban), és keresd a legnagyobb összeget. Meddig mehet a kezdőnap indexe?

6. fejezet

- **6.2.** Egy sor hat száma egyszerre beolvasható: `while (be >> o1 >> p1 >> m1 >> o2 >> p2 >> m2)`. A ciklusban készíts egy `Hivas` változót az `mpben` függvénnyel, és tedd a vektorba.
- **6.3.** Maximumkiválasztás a `hossz()` értékek szerint. A perc: `hossz / 60`, a másodperc: `hossz % 60`.
- **6.4.** Az óra: `ossz / 3600`, a perc: `ossz % 3600 / 60`, a másodperc: `ossz % 60`.
- **6.5.** `std::vector<int> darab(24);` – minden hívásnál `darab[kezdet / 3600]++`; A fájlba csak azokat az órákat írd ki, ahol a darabszám nem nulla.
- **6.6.** Megszámolás: az `i`-edik hívás `kezdet` mezőjét hasonlítsd az `i - 1`-edik `veg` mezőjéhez. A ciklus 1-től induljon.
- **6.7.** Maximumkiválasztás a `hivasok[i].kezdet - hivasok[i - 1].veg` különbségekre.
- **6.8.** A kezdet órája `kezdet / 3600`, a perce `kezdet % 3600 / 60`.

7. fejezet

- **7.1.** A fejléchez egy `std::getline(be, fejlec);` a ciklus előtt. A `feldolgoz` függvényben öt `std::getline(ss, ..., ';')`; a három számot `std::stoi` alakítja át.
- **7.2.** A perc `mp / 60`, a másodperc `mp % 60`. Építsd fel a szöveget `std::to_string` segítségével; ha a másodperc 10-nél kisebb, fűzz elé egy `"0"` karaktert.
- **7.3.** Minimumkiválasztás az `ido()` szerint.
- **7.4.** Három számláló, vagy egy függvény: `int letszam(std::vector<Futo> v, std::string kat)`.
- **7.5.** Minimumkiválasztás, de csak az adott kategória versenyzői között. Vigyázz: a kezdőérték is ebből a kategóriából való legyen (például indulj -1 indexről, és az első megfelelő versenyzőt fogadd el).
- **7.6.** Az átlag `double`; az `idoSzoveg` függvénynek az egész részét add át.
- **7.7.** Keresés a rajtszám alapján (`hely = -1` kezdőértékkel). A helyezéshez számold meg, hány versenyző ideje jobb az övéénél; a helyezés ennél eggyel nagyobb.
- **7.8.** Rendezés összehasonlító függvénnyel (`return a.ido() < b.ido();`), utána egy ciklus ír a fájlba.
- **7.9.** A már rendezett vektoron menj végig kategóriánként, és az első három megfelelő versenyzőt írd ki. Számláló kell hozzá.

8. fejezet

- **8.1.** A `feldolgoz` függvényben négy `std::getline(ss, ..., ';')`, a két számot `std::stoi` alakítja át.
- **8.4.** A függvényben feltételes összegzés: csak azokat a tételeket add össze, amelyeknek a `nap` mezője egyezik. A napok nevét tedd egy vektorba, és ciklussal hívd meg rájuk a függvényt.
- **8.5.** Egy terméknevet csak akkor tegyél az új vektorba, ha még nincs benne (előző feladatlap, 5.6. és 9.4. feladat).
- **8.6.** Készíts minden terméknevhez egy `Osszesites` elemet nullákkal. Írj `int keres(std::vector<Osszesites> v, std::string nev)` függvényt, amely megadja a termék indexét. Menj végig a tételeken: keresd meg a termék indexét, és növeld annak az elemnek a `darab` és a `bevetel` mezőjét.
- **8.7.** Rendezés a `bevetel` mező szerint, kiírás `;` jelekkel. A visszaolvasáshoz új `feldolgoz` függvény kell az `Osszesites` structhoz.
- **8.8.** Minden termékre dönts el (eldöntés), volt-e olyan tétel, amelynek a napja `pentek`, a terméke pedig az adott termék. Azokat írd ki, amelyekre nem volt.